Create authentication service that returns JWT

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.cognizant</groupId>
  <artifactId>spring-learn</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name/>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
```

```
      <version>0.11.5</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>
```

AuthenticationResponse.java

```
package com.cognizant.springlearn.model;

public class AuthenticationResponse {

   private String token;

   public AuthenticationResponse() {

   }

   public AuthenticationResponse(String token) {

      this.token = token;

   }

   public String getToken() {

      return token;

   }

  public void setToken(String token) {
```

```
        this.token = token;
    }
}
```

JwtUtil.java

```
package com.cognizant.springlearn.util;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import java.util.Date;
public class JwtUtil {
    private static final String SECRET =
            "mysupersecretkeymysupersecretkey";
    public static String generateToken(String username) {
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(
                        new Date(System.currentTimeMillis() + 3600000))
                .signWith(
                        io.jsonwebtoken.security.Keys.hmacShaKeyFor(
                                SECRET.getBytes()),
                        SignatureAlgorithm.HS256)
                .compact();
    }
}
```

AuthenticationController.java

```
package com.cognizant.springlearn.controller;
import com.cognizant.springlearn.model.AuthenticationResponse;
import com.cognizant.springlearn.util.JwtUtil;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
@RestController
public class AuthenticationController {
   @GetMapping("/authenticate")
   public AuthenticationResponse authenticate(
         @RequestHeader("Authorization") String authHeader) {
      String base64Credentials =
            authHeader.substring("Basic ".length());
      byte[] decoded =
            Base64.getDecoder().decode(base64Credentials);
      String credentials =
            new String(decoded, StandardCharsets.UTF_8);
      String[] values = credentials.split(":", 2);
      String username = values[0];
      String password = values[1];
      String token = JwtUtil.generateToken(username);
      return new AuthenticationResponse(token);
   }
}
```

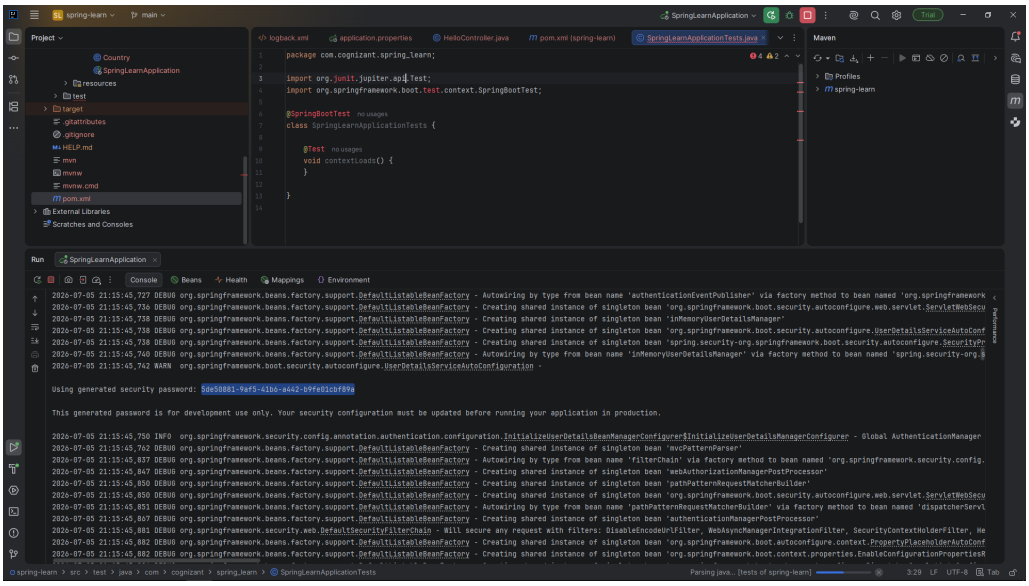
SecurityConfig.java

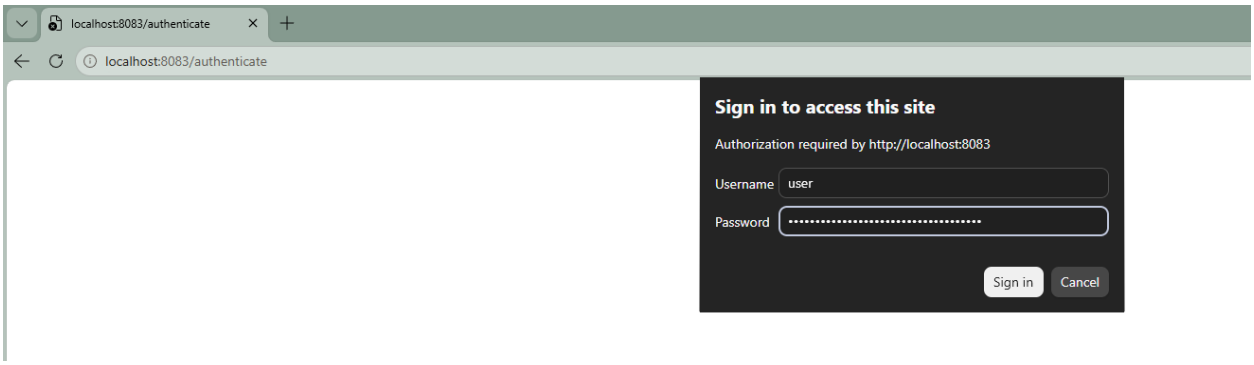
```
package com.cognizant.springlearn.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
```

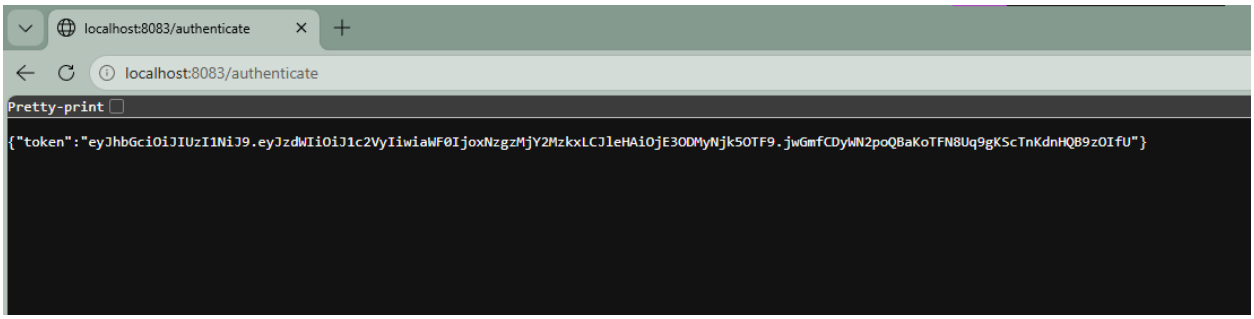
```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.web.SecurityFilterChain;

@Configuration

public class SecurityConfig {

   @Bean

   SecurityFilterChain filterChain(HttpSecurity http)

         throws Exception {  http

            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth ->

                  auth

                        .requestMatchers("/authenticate")

                        .permitAll()

                        .anyRequest()

                        .authenticated())

            .httpBasic(Customizer.withDefaults());

      return http.build();

   }

}
```

SpringLearnApplication.java

```
package com.cognizant.spring_learn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

   public static void main(String[] args) {
     SpringApplication.run(SpringLearnApplication.class, args);
   }

}
```


spring-learn
main
SpringLearnApplication
Project
logback.xml
application.properties
HelloController.java
pom.xml (spring-learn)
SpringLearnApplicationTests.java
Maven
Profiles
spring-learn
package com.cognizant.spring_learn;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
@SpringBootTest
class SpringLearnApplicationTests {
@Test
void contextLoads() {
}
}
Run
SpringLearnApplication
Console
Beans
Health
Mappings
Environment
Using generated security password: 9de50881-9af5-41b6-a442-b9fe01cbf89a
This generated password is for development use only. Your security configuration must be updated before running your application in production.


localhost:8083/authenticate
localhost:8083/authenticate
Sign in to access this site
Authorization required by http://localhost:8083
Username
user
Password
Sign in
Cancel


localhost:8083/authenticate
localhost:8083/authenticate
Pretty-print
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzMjY2MzkxLCJleHAiOjE3ODMyNjk5OTF9.jwGmfCDyWN2poQBaKoTFN8Uq9gKScTnKdnHQB9zOIfU"}